

Ciencia de Datos

Clasificación de sentimiento en comentarios estudiantiles

Comparación de TF-IDF con Regresión Logística y BETO

Informe final

Integrantes: Diego Patiño
María José Vire

Docente: Ing. Jorge Maldonado

Asignatura: Ciencia de Datos

4 de octubre de 2026

1. Introducción

Los comentarios estudiantiles ofrecen información sobre experiencias y percepciones relacionadas con la actividad universitaria. Su clasificación automática por sentimiento permite organizar esta información, aunque no reemplaza la interpretación del contexto educativo ni la revisión de los casos ambiguos.

Este proyecto estudió una tarea supervisada de clasificación binaria de comentarios en español. Se comparó un enfoque de representación léxica, TF-IDF con Regresión Logística, con BETO, un modelo Transformer preentrenado en español. Ambos enfoques utilizaron el mismo predictor textual y las mismas particiones de datos. Se separaron el entrenamiento, la selección mediante validación y la evaluación final sobre prueba, con el propósito de evitar que los resultados finales influyeran en las decisiones del modelado.

2. Objetivo

Clasificar cada comentario como Negativo (0) o Positivo (1), a partir exclusivamente de `comentario_limpio`, y comparar el rendimiento de un modelo base clásico y BETO mediante F1-macro como métrica principal. La variable objetivo fue `sentimiento_id`. Se complementó la evaluación con accuracy, precisión y recall macro, métricas por clase y matrices de confusión.

3. Dataset y preparación de datos

Se utilizó el *Student Feedback Sentiment Analysis Dataset*, versión 1.0.1, publicado por Anabel Pineda-Briseño y Jimmy Oblitas en Zenodo, registro 18797217 [1]. El archivo original contenía 23 168 registros y las variables `comentario`, `Valor` y `nombreAspecto`.

La preparación preservó el contenido lingüístico: únicamente se normalizaron secuencias de espacios en blanco y se eliminaron espacios al inicio y al final. Se conservaron mayúsculas, tildes, negaciones, signos y números. Se excluyeron dos registros con el marcador no interpretable `##¿NOMBRE?`, ocho observaciones con etiquetas contradictorias para textos normalizados equivalentes y 35 duplicados excedentes con etiquetas consistentes. Para estos últimos se retuvo el menor índice original. En total se retiraron 45 registros y se conservaron 23 123 observaciones.

Se realizó una partición estratificada 70/15/15 con semilla 42. Se verificó la ausencia de índices y textos normalizados compartidos entre particiones. `nombreAspecto` no fue predictor; `Valor` se utilizó para codificar las etiquetas y `indice_original` se mantuvo para trazabilidad.

Tabla 1: Particiones congeladas, compartidas por ambos enfoques.

Partición	Registros	Negativo (0)	Positivo (1)
Train	16 186	6 870	9 316
Validation	3 468	1 472	1 996
Test	3 469	1 473	1 996
Total	23 123	9 815	13 308

4. Modelo base

El modelo base fue un pipeline de scikit-learn con `TfidfVectorizer` y `LogisticRegression` [2, 3, 4]. Tanto el vocabulario como los coeficientes del clasificador se ajustaron únicamente con train; validation se utilizó para comparar configuraciones. No se reajustó el modelo sobre la unión de train y validation antes de la evaluación final.

El notebook de datos documentó seis configuraciones que variaron el rango de n-gramas y el parámetro de regularización C . Se seleccionó C1, basada en unigramas y $C = 1,0$. El vectorizador utilizó `min_df=2`, `max_features=20000`, conversión a minúsculas, normalización L2 y suavizado de IDF. No se eliminaron stop words ni tildes. El vocabulario ajustado de C1 contuvo 5653 características.

La Regresión Logística empleó penalización L2, solver `liblinear`, un máximo de 1000 iteraciones, semilla 42 y sin pesos de clase. En validation, C1 obtuvo **F1-macro = 0.8757** y **accuracy = 0.8789**. Se conservó el pipeline ajustado para la prueba final, en lugar de volver a entrenarlo después de seleccionar la configuración.

5. Modelo Transformer

Se utilizó BETO cased, identificado como `dccuchile/bert-base-spanish-wwm-cased`, mediante Hugging Face Transformers 4.57.1. El tokenizer y el modelo compartieron una revisión fijada. Cada experimento partió del modelo preentrenado original, con una cabeza de clasificación binaria, y no de un checkpoint afinado de otro experimento. Se usaron dos etiquetas con el mapeo Negativo = 0 y Positivo = 1.

5.1. Tokenización y longitud máxima

La longitud máxima se determinó exclusivamente con los 16186 comentarios de train. Se tokenizaron sin truncamiento ni padding, incluyendo los tokens especiales. La mediana fue de 14 tokens, el percentil 95 de 52 y el percentil 99 de 103.15. La regla previa al análisis consistió en elegir el menor candidato de {64, 128, 256, 512} que cubriera al menos el 99 % de train.

Tabla 2: Cobertura de las longitudes candidatas en train.

Límite (tokens)	Cobertura (%)	Secuencias que excedieron el límite
64	96.9480	494
128	99.3822	100
256	99.8950	17
512	99.9938	1

Se fijó `max_length=128`: fue el menor candidato que satisfizo la regla y dejó un 0.6178 % de secuencias de train por encima del límite. Durante el entrenamiento y la evaluación se aplicó truncamiento a esa longitud y padding dinámico por batch. Validation y test no participaron en esta decisión. Los únicos datos predictivos fueron el texto tokenizado y su etiqueta; los índices y los hashes se conservaron como metadatos separados.

6. Diseño experimental

Se realizaron tres experimentos independientes de BETO, diferenciados únicamente por la tasa de aprendizaje: T1 (1×10^{-5}), T2 (2×10^{-5}) y T3 (3×10^{-5}). Los tres utilizaron las mismas particiones, revisión del modelo, longitud máxima y reglas de evaluación.

Tabla 3: Configuración común del entrenamiento Transformer.

Parámetro	Valor
Semilla	42
Épocas	3
Longitud máxima	128 tokens
Weight decay	0.01
Batch físico / acumulación / efectivo	16 / 1 / 16
Dispositivo y precisión	GPU Tesla T4; FP16
Optimizador y scheduler	AdamW de PyTorch; scheduler lineal; cero pasos de warmup
Evaluación y guardado	Al finalizar cada época
Pesos de clase / oversampling / early stopping	No aplicados

Se conservaron los tres checkpoints de cada experimento, uno por época. Se desactivó la selección automática del mejor modelo al terminar el entrenamiento (`load_best_model_at_end=False`); la selección se efectuó explícitamente con la regla del proyecto, sobre validation. La interfaz de entrenamiento utilizó `Trainer` y las opciones de evaluación por época de Transformers [6].

7. Selección mediante validación

La regla priorizó el F1-macro de validation redondeado a cuatro decimales. En caso de empate, se compararon accuracy, menor learning rate y, finalmente, época más temprana. La regla se aplicó para seleccionar la época de cada experimento y el ganador global. El conjunto de prueba no intervino en ninguna de estas decisiones.

Tabla 4: Resultados de validación comunicados para las configuraciones de BETO.

Experimento	Learning rate	F1-macro	Accuracy
T1	1×10^{-5}	0.9141	0.9158
T2	2×10^{-5}	0.9164	0.9181
T3	3×10^{-5}	0.9100	0.9118

La selección quedó fijada en **T2**, learning rate 2×10^{-5} , **época 3** y **checkpoint-3036**. Estos identificadores están registrados en el JSON de evaluación final y en el notebook de BETO. La elección se cerró antes de la evaluación predictiva de test.

Alcance de la evidencia: la tabla recoge los valores oficiales comunicados por el equipo. Los JSON individuales de T1/T2/T3 y el resumen de selección no están versionados ni disponibles en la copia local. El notebook conservado tampoco contiene outputs con esos seis valores, por lo que no pudieron corroborarse en archivos. Las métricas de C1 en validation y las de ambos modelos en test sí se verificaron en sus JSON. No se reconstruyeron resultados faltantes.

8. Evaluación final sobre test

Se evaluaron ambos modelos congelados sobre las mismas 3 469 observaciones de test. El predictor fue `comentario_limpio` y el objetivo, `sentimiento_id`. Ambos resultados registraron el mismo SHA-256 del CSV y de sus metadatos. La interfaz exigió confirmación explícita y un marcador por intento para impedir repeticiones accidentales en el mismo directorio.

Para BETO se cargó exclusivamente `checkpoint-3036` de T2. La evaluación se realizó en CPU con FP32, sin autocast FP16, con `torch.inference_mode()` y el modelo en modo de evaluación. El paso de GPU/FP16 durante el entrenamiento a CPU/FP32 para inferencia no modificó los pesos ni el checkpoint seleccionado. C1 se cargó desde su pipeline serializado, sin llamar a `fit`. Los dos JSON registraron una única evaluación predictiva, sin entrenamiento ni reelección.

F1-macro se calculó como la media no ponderada del F1 de las dos clases [5]. La accuracy midió la proporción de aciertos. Precisión y recall macro se promediaron igualmente entre clases, con `zero_division=0`. Para BETO, las predicciones se obtuvieron mediante `argmax` de logits, sin ajustar umbrales con test.

9. Comparación de resultados

Tabla 5: Evaluación final sobre test. Deltas calculados como BETO menos C1.

Métrica	Baseline C1	BETO T2	Diferencia
Accuracy	0.8694	0.9092	+0.0398
F1-macro	0.8662	0.9073	+0.0411
Precisión macro	0.8667	0.9064	+0.0397
Recall macro	0.8658	0.9085	+0.0427
F1 Negativo	0.8456	0.8942	—
F1 Positivo	0.8869	0.9205	—

BETO T2 obtuvo el mayor F1-macro en este conjunto de prueba: 0.9073 frente a 0.8662 de C1. La diferencia fue de +0.0411 en escala 0–1, equivalente aproximadamente a 4.11 puntos porcentuales. Las cuatro métricas globales y el F1 de ambas clases fueron mayores para BETO. Los deltas se calcularon con los valores completos de los JSON y se redondearon a cuatro decimales para su presentación. Esta comparación fue descriptiva y no constituyó una nueva selección.

Tabla 6: Matrices de confusión en test: filas reales y columnas predichas, orden [0, 1].

Baseline C1			BETO T2		
Clase real	Pred. 0	Pred. 1	Clase real	Pred. 0	Pred. 1
Negativo (0)	1240	233	Negativo (0)	1331	142
Positivo (1)	220	1776	Positivo (1)	173	1823

C1 cometió 453 errores y BETO 315: 138 errores menos en términos agregados. Los negativos clasificados como positivos disminuyeron de 233 a 142, y los positivos clasificados como negativos, de 220 a 173. Los recuentos no identifican qué ejemplos se corrigieron. El F1 de Negativo fue menor que el de Positivo en ambos modelos; no se atribuye esta diferencia a una causa específica sin analizar los errores.

10. Conclusiones

En el conjunto de prueba reservado, BETO T2 presentó un rendimiento predictivo mayor que el modelo base TF-IDF con Regresión Logística. La ventaja se observó en F1-macro, accuracy, precisión macro, recall macro y F1 de cada clase. La disminución agregada de errores afectó a ambos tipos de confusión, por lo que la mejora no se limitó a una sola clase.

La separación entre entrenamiento, validación y prueba permitió distinguir la elección del modelo de la descripción de su rendimiento final. T2 y su checkpoint de la tercera época se fijaron mediante validation; después de observar test no se reentrenó, no se ajustaron hiperparámetros y no se seleccionó otro checkpoint. La inferencia en CPU reutilizó los parámetros aprendidos durante el entrenamiento en GPU.

Las conclusiones se limitan a este dataset, su preparación y la partición utilizada. No se evaluaron otros dominios o instituciones, ni se estimaron intervalos de confianza o significación estadística de las diferencias. No se puede afirmar generalización fuera del corpus ni superioridad universal de BETO. Los recuentos tampoco explican por sí solos los fenómenos lingüísticos de los errores. La exclusión de contradicciones y duplicados delimitó la población estudiada.

11. Reproducibilidad

El entrenamiento y la selección utilizaron el commit

`90afa082979bbe396f5a6a4906cadee9c351dd4f`

La evaluación final de ambos modelos registró el commit

`ddc3d2e51246e8450a0d000cd48af55bf101f878`

Los JSON finales declararon un árbol de trabajo limpio. La configuración y la revisión de BETO se verificaron antes de cargar el checkpoint. La revisión del modelo fue

`c4d86612f51b4f46759c8390d1798c2febe71b93`

El SHA-256 compartido de test fue:

`f1e7c5da7760aaf3ee0f89c3d86d1307db5a347cb1b543f551276b12e9c7851b`

La reproducción requiere las particiones congeladas, sus metadatos, el pipeline C1 y los archivos del checkpoint seleccionado. Los datos procesados y los modelos no se redistribuyeron en el repositorio. Los hashes identifican los artefactos concretos; la semilla 42 y las versiones fijadas apoyan la reproducibilidad, sin asegurar determinismo total del entrenamiento entre entornos.

La inferencia de BETO registró Python 3.13.15, PyTorch 2.11.0+cpu y Transformers 4.57.1. C1 registró Python 3.12.3, scikit-learn 1.6.1, joblib 1.6.0 y NumPy 2.1.3. El tiempo medido de predicción y cálculo de métricas fue de 876.8544 s para BETO y 0.0417 s para C1. Excluyó la verificación previa y la carga de modelos. Dado que las ejecuciones se realizaron en entornos distintos y sin un benchmark controlado de hardware, estos tiempos se presentan como trazabilidad, no como una comparación normalizada de eficiencia.

Las fuentes internas fueron los notebooks de datos y preparación, el cuaderno `03_BETO_reproducible_FINAL.ipynb`, la configuración, los metadatos y los JSON `transformer_T2_test.json`, `baseline_C1_test.json` y `comparison_test.json`. El informe se elaboró sin repetir entrenamiento ni predicciones sobre test.

Referencias

- [1] Pineda-Briseño, A., y Oblitas, J. (s. f.). *Student Feedback Sentiment Analysis Dataset* (versión 1.0.1). Zenodo. <https://doi.org/10.5281/zenodo.18797217>
- [2] scikit-learn. (s. f.). *TfidfVectorizer*. Documentación oficial. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- [3] scikit-learn. (s. f.). *LogisticRegression*. Documentación oficial. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [4] scikit-learn. (s. f.). *Pipeline*. Documentación oficial. <https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>
- [5] scikit-learn. (s. f.). *f1_score*. Documentación oficial. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
- [6] Hugging Face. (s. f.). *Trainer*. Documentación oficial de Transformers, versión 4.57.1. https://huggingface.co/docs/transformers/v4.57.1/en/main_classes/trainer